

- 快速入门
- 什么是 GoClaw?
- 安装
- 快速开始
- 配置
- Web Dashboard 导览
- 从 OpenClaw 迁移
- 核心概念
- GoClaw 工作原理
- Agent 详解
- Session 与历史记录
- Tools 概览
- Memory 系统
- 多租户
- AGENT
- 创建 Agent
- Open vs. Predefined Agent
- Context 文件
- Summoning 与 Bootstrap
- 编辑 Agent Personality
- 共享与访问控制
- 用户覆盖
- System Prompt 结构
- PROVIDER
- Provider 概览
- Anthropic (Claude)
- OpenAI / Azure OpenAI
- OpenRouter
- Google Gemini
- DeepSeek
- Groq
- Mistral
- xAI (Grok)
- MiniMax
- Cohere
- Perplexity
- DashScope (阿里通义千问)
- Vertex AI (Google Cloud)
- 百炼
- Zai
- Claude CLI
- Codex / ChatGPT (OAuth)
- Ollama
- Ollama Cloud
- Suno
- YesScale
- Novita AI

翻译自 [English version](#)

REST API

agent 管理、provider、skill、traces 等所有 `/v1` HTTP 端点。

概览

需要完整索引？ 查看 [API 端点目录](#) — 自动生成的全部 ~308 REST 端点列表。

GoClaw 的 HTTP API 与 WebSocket gateway 共用同一端口。所有端点需要在 `Authorization` 头中提供与 `GOCLAW_GATEWAY_TOKEN` 匹配的 `Bearer` token。

交互式文档： `/docs` (Swagger UI) · 原始规范： `/v1/openapi.json`

Base URL: `http://<host>:<port>`

认证头：

`Authorization: Bearer YOUR_GATEWAY_TOKEN`

用户身份头（可选，用于按用户范围隔离）：

`X-GoClaw-User-Id: user123`

通用请求头

请求头	用途
<code>Authorization</code>	Bearer token
<code>X-GoClaw-User-Id</code>	多租户上下文的外部用户 ID
<code>X-GoClaw-Agent-Id</code>	范围操作的 agent 标识符
<code>X-GoClaw-Tenant-Id</code>	租户范围——UUID 或 slug
<code>Accept-Language</code>	国际化错误消息的语言（ <code>en</code> 、 <code>vi</code> 、 <code>zh</code> ）
<code>X-GoClaw-No-Image-Gen</code>	（可选）在该请求中发送此头以 opt-out 原生图片生成。绕过 provider capability、agent flag 及 tri-level gate。适用于 chat 端点。

输入验证： 所有字符串输入均经过净化——ILIKE 查询中 SQL 特殊字符会被转义，请求体限制为 1 MB，agent/provider/tool 名称通过白名单模式（`[a-zA-Z0-9_-]`）验证。

聊天补全

OpenAI 兼容的聊天 API，用于以编程方式访问 agent。

`POST /v1/chat/completions`

```
curl -X POST http://localhost:18790/v1/chat/completions \
-H "Authorization: Bearer TOKEN" \
-H "Content-Type: application/json" \
-d '{
  "model": "goclaw:agent-id-or-key",
  "messages": [{"role": "user", "content": "Hello"}],
}
```

- 快速入门
- 什么是 GoClaw?
- 安装
- 快速开始
- 配置
- Web Dashboard 导览
- 从 OpenClaw 迁移
- 核心概念
- GoClaw 工作原理
- Agent 详解
- Session 与历史记录
- Tools 概览
- Memory 系统
- 多租户

- AGENT
- 创建 Agent
- Open vs. Predefined Agent
- Context 文件
- Summoning 与 Bootstrap
- 编辑 Agent Personality
- 共享与访问控制
- 用户覆盖
- System Prompt 结构
- PROVIDER
- Provider 概览
- Anthropic (Claude)
- OpenAI / Azure OpenAI
- OpenRouter
- Google Gemini
- DeepSeek
- Groq
- Mistral
- xAI (Grok)
- MiniMax
- Cohere
- Perplexity
- DashScope (阿里通义千问)
- Vertex AI (Google Cloud)
- 百炼
- Zai
- Claude CLI
- Codex / ChatGPT (OAuth)
- Ollama
- Ollama Cloud
- Suno
- YesScale
- Novita AI

响应（非流式）：

```
{
  "id": "chatcmpl-...",
  "object": "chat.completion",
  "choices": [{
    "index": 0,
    "message": {"role": "assistant", "content": "..."},
    "finish_reason": "stop"
  }],
  "usage": {"prompt_tokens": 10, "completion_tokens": 20, "total_tokens": 30}
}
```

设置 `"stream": true` 可获取以 `data: [DONE]` 结尾的 SSE 数据块。

OpenResponses 协议

POST /v1/responses

基于响应的替代协议（与 OpenAI Responses API 兼容）。接受相同的认证方式，返回结构化响应对象。

会话（Sessions）

通过 HTTP 进行只读的会话发现与分支，适用于无法保持 WebSocket 连接的自动化场景。非管理员调用方仅能看到自己的会话（按 `X-GoClaw-User-Id` 匹配）；管理员和 gateway-owner 令牌可看到所有会话。

完整的会话生命周期（preview、patch、delete、reset、compact）通过 **WebSocket RPC** 提供。这些 HTTP 路由用于发现、分支和增量读取历史。

GET /v1/sessions

分页列出会话。

```
curl "http://localhost:18790/v1/sessions?agent_id=AGENT_UUID&limit=20&offset=0" \
-H "Authorization: Bearer TOKEN" \
-H "X-GoClaw-User-Id: user123"
```

查询参数：

参数	类型	说明
<code>agent_id</code>	string	按 agent 过滤（别名： <code>agentId</code> ）
<code>channel</code>	string	按 channel 过滤
<code>limit</code>	integer	每页数量（默认 20）
<code>offset</code>	integer	页偏移（默认 0）

响应： `{ "sessions": [...], "total": N, "limit": 20, "offset": 0 }`

POST /v1/chat/sessions/{key}/branch

从已有会话分支出一个新会话，复制消息到指定索引。路径段 `{key}` 是源会话键；分支复用相同的 agent 键。

```
curl -X POST http://localhost:18790/v1/chat/sessions/{key}/branch \
-H "Authorization: Bearer TOKEN" \
-H "Content-Type: application/json" \
-d '{
```

- 快速入门
- 什么是 GoClaw?
- 安装
- 快速开始
- 配置
- Web Dashboard 导览
- 从 OpenClaw 迁移
- 核心概念
- GoClaw 工作原理
- Agent 详解
- Session 与历史记录
- Tools 概览
- Memory 系统
- 多租户

- AGENT
- 创建 Agent
- Open vs. Predefined Agent
- Context 文件
- Summoning 与 Bootstrap
- 编辑 Agent Personality
- 共享与访问控制
- 用户覆盖
- System Prompt 结构

- PROVIDER
- Provider 概览
- Anthropic (Claude)
- OpenAI / Azure OpenAI
- OpenRouter
- Google Gemini
- DeepSeek
- Groq
- Mistral
- xAI (Grok)
- MiniMax
- Cohere
- Perplexity
- DashScope (阿里通义千问)
- Vertex AI (Google Cloud)
- 百炼
- Zai
- Claude CLI
- Codex / ChatGPT (OAuth)
- Ollama
- Ollama Cloud
- Suno
- YesScale
- Novita AI

```
"up_to_index": 8,
"new_session_key": "",
"label": "what-if branch",
"metadata": {}
}'
```

字段	类型	说明
up_to_index	integer	必填。将消息 [0, up_to_index) 复制到分支
new_session_key	string	可选。必须与源使用相同的 agent 键。省略时自动生成
label	string	可选的分支标签（供人阅读）
metadata	object	可选的字符串键值元数据

响应（ 201 Created ）：

```
{
  "ok": true,
  "source_key": "agent:researcher:direct:...",
  "session_key": "agent:researcher:branch:direct:...",
  "copied_messages": 8,
  "total_messages": 20,
  "label": "what-if branch"
}
```

若目标键已存在返回 409，若 up_to_index 越界返回 400。

GET /v1/chat/sessions/{key}/history/follow

基于游标的增量历史读取 — 轮询新消息而无需重新获取整个对话记录。返回消息中的文件和媒体引用会被签名。

查询参数：

参数	类型	说明
cursor	integer	起始读取索引（默认 0）
limit	integer	返回消息数上限（默认 50，最大 200）

响应：

```
{
  "session_key": "agent:researcher:direct:...",
  "cursor": 0,
  "next_cursor": 50,
  "total": 120,
  "messages": [ ... ],
  "reset": false,
  "updated": "2026-06-07T08:00:00Z"
}
```

将 next_cursor 作为下一次的 cursor 继续读取。reset: true（且 messages 为空数组）表示游标已超出当前历史长度 — 从 0 重新开始。

Agent

agent 管理的 CRUD 操作。多租户上下文需要 X-GoClaw-User-Id 头。

GET /v1/agents

列出所有 agent。

```
curl http://localhost:18790/v1/agents \
```

- 快速入门
- 什么是 GoClaw?
- 安装
- 快速开始
- 配置
- Web Dashboard 导览
- 从 OpenClaw 迁移

- 核心概念
- GoClaw 工作原理
- Agent 详解
- Session 与历史记录
- Tools 概览

- Memory 系统
- 多租户

- AGENT
- 创建 Agent
- Open vs. Predefined Agent
- Context 文件
- Summoning 与 Bootstrap
- 编辑 Agent Personality
- 共享与访问控制
- 用户覆盖
- System Prompt 结构

- PROVIDER
- Provider 概览
- Anthropic (Claude)
- OpenAI / Azure OpenAI
- OpenRouter
- Google Gemini
- DeepSeek
- Groq
- Mistral
- xAI (Grok)
- MiniMax
- Cohere
- Perplexity

- DashScope (阿里通义千问)
- Vertex AI (Google Cloud)
- 百炼
- Zai
- Claude CLI
- Codex / ChatGPT (OAuth)
- Ollama
- Ollama Cloud
- Suno
- YesScale
- Novita AI

POST /v1/agents

创建新 agent。

```
curl -X POST http://localhost:18790/v1/agents \
-H "Authorization: Bearer TOKEN" \
-H "Content-Type: application/json" \
-d '{
  "agent_key": "researcher",
  "display_name": "Research Assistant",
  "agent_type": "open",
  "provider": "anthropic",
  "model": "claude-sonnet-4-5-20250929",
  "context_window": 200000,
  "max_tool_iterations": 20,
  "workspace": "~/goclaw/workspace-researcher"
}'
```

GET /v1/agents/{id}

按 ID 获取单个 agent。

PUT /v1/agents/{id}

更新 agent。只需发送要修改的字段。

DELETE /v1/agents/{id}

删除 agent。

POST /v1/agents/{id}/regenerate

从模板重新生成 agent context 文件。

POST /v1/agents/{id}/resummon

为 predefined agent 重新触发基于 LLM 的 summoning。

POST /v1/agents/{id}/cancel-summon

强制中止卡住的 summoning 进程。将处于 `summoning` 状态的 agent 转换为 `summon_failed`，以便重新配置或重新触发。如果 agent 不在 `summoning` 状态，返回 `409`。

Agent 共享

方法	路径	说明
GET	/v1/agents/{id}/shares	列出 agent 的共享记录
POST	/v1/agents/{id}/shares	与用户共享 agent
DELETE	/v1/agents/{id}/shares/{userID}	撤销共享

Predefined Agent 实例

方法	路径	说明
GET	/v1/agents/{id}/instances	列出用户实例
GET	/v1/agents/{id}/instances/{userID}/files	列出用户 context 文件
PUT	/v1/agents/{id}/instances/{userID}/files/{fileName}	更新用户 context 文件（管理员）
PATCH	/v1/agents/{id}/instances/{userID}/metadata	更新实例元数据
GET	/v1/agents/{id}/system-prompt-preview	预览已渲染的 system prompt（管理员）

如需读取文件内容，请先通过 `GET /v1/agents/{id}/instances/{userID}/files` 列出文件，再通过 `Vault` 或 `Storage` API 获取。不存在单文件 GET 的实例文件端点。

以 tar.gz 归档格式导出和导入 agent 配置及数据。支持按 section 选择性导出。

- 快速入门
- 什么是 GoClaw?
- 安装
- 快速开始
- 配置
- Web Dashboard 导览
- 从 OpenClaw 迁移

- 核心概念
- GoClaw 工作原理
- Agent 详解
- Session 与历史记录
- Tools 概览
- Memory 系统
- 多租户

- AGENT
- 创建 Agent
- Open vs. Predefined Agent
- Context 文件
- Summoning 与 Bootstrap
- 编辑 Agent Personality
- 共享与访问控制
- 用户覆盖
- System Prompt 结构

- PROVIDER
- Provider 概览
- Anthropic (Claude)
- OpenAI / Azure OpenAI
- OpenRouter
- Google Gemini
- DeepSeek
- Groq
- Mistral
- xAI (Grok)
- MiniMax
- Cohere
- Perplexity
- DashScope (阿里通义千问)
- Vertex AI (Google Cloud)
- 百炼
- Zai
- Claude CLI
- Codex / ChatGPT (OAuth)
- Ollama
- Ollama Cloud
- Suno
- YesScale
- Novita AI

方法	路径	说明
GET	/v1/agents/{id}/export/preview	预览各 section 数量（不生成归档）
GET	/v1/agents/{id}/export	直接下载 agent 归档（tar.gz）
GET	/v1/agents/{id}/export/download/{token}	通过短效 token 下载已准备好的归档（5 分钟有效）
POST	/v1/agents/import	将归档导入为 新 agent （multipart 字段 <code>file</code> ）
POST	/v1/agents/import/preview	解析归档并返回 manifest，不执行导入
POST	/v1/agents/{id}/import	将归档数据 合并 到已有 agent

导出查询参数：

参数	类型	说明
sections	string	逗号分隔的 section 列表，默认 <code>config,context_files</code> 。可选： <code>config</code> 、 <code>context_files</code> 、 <code>memory</code> 、 <code>knowledge_graph</code> 、 <code>cron</code> 、 <code>user_profiles</code> 、 <code>user_overrides</code> 、 <code>workspace</code>
stream	bool	为 <code>true</code> 时以 SSE 流式推送进度，最后发送含 <code>download_url</code> 的 <code>complete</code> 事件

导入响应（`201 Created`）：

```
{
  "agent_id": "uuid",
  "agent_key": "researcher",
  "context_files": 3,
  "memory_docs": 12,
  "kg_entities": 50,
  "kg_relations": 30
}
```

Cron 作业始终以**禁用**状态导入。同名作业将被跳过。归档大小上限：500 MB。

GET /v1/agents/{id}/codex-pool-activity

返回使用 **Codex OAuth pool** 的 agent 的路由活动和每账户健康状态。要求 agent 的 provider 为 `chatgpt_oauth` 类型并已配置 pool。

认证：需要 Bearer token。请求用户必须有权访问该 agent。

查询参数：

参数	类型	默认值	说明
limit	integer	18	返回的最近请求数（最大 50）

响应中 **strategy** 的取值：

取值	说明
round_robin	均匀轮询分发
priority_order	按配置顺序优先选择 provider（默认）

重大变更（客户端影响）：Codex 账号池 API 响应中，对于原本返回 `primary_first` / `manual` 的相同路由配置，现已改为返回 `priority_order`。请求体仍接受旧值以保持向后兼容。请更新所有按字面比较 `strategy` 字符串的客户端代码。

响应：

```
{
  "strategy": "priority_order",
```

快速入门

什么是 GoClaw?

安装

快速开始

配置

Web Dashboard 导览

从 OpenClaw 迁移

核心概念

GoClaw 工作原理

Agent 详解

Session 与历史记录

Tools 概览

Memory 系统

多租户

AGENT

创建 Agent

Open vs. Predefined Agent

Context 文件

Summoning 与 Bootstrap

编辑 Agent Personality

共享与访问控制

用户覆盖

System Prompt 结构

PROVIDER

Provider 概览

Anthropic (Claude)

OpenAI / Azure OpenAI

OpenRouter

Google Gemini

DeepSeek

Groq

Mistral

xAI (Grok)

MiniMax

Cohere

Perplexity

DashScope (阿里通义千问)

Vertex AI (Google Cloud)

百炼

Zai

Claude CLI

Codex / ChatGPT (OAuth)

Ollama

Ollama Cloud

Suno

YesScale

Novita AI

```
"pool_providers": ["openai-codex", "codex-work"],
"stats_sample_size": 24,
"provider_counts": [
  {
    "provider_name": "openai-codex",
    "request_count": 14,
    "direct_selection_count": 10,
    "failover_serve_count": 4,
    "success_count": 13,
    "failure_count": 1,
    "consecutive_failures": 0,
    "success_rate": 92,
    "health_score": 88,
    "health_state": "healthy",
    "last_used_at": "2026-03-27T08:00:00Z"
  }
],
"recent_requests": [
  {
    "span_id": "uuid",
    "trace_id": "uuid",
    "started_at": "2026-03-27T08:00:00Z",
    "status": "success",
    "duration_ms": 1240,
    "provider_name": "openai-codex",
    "selected_provider": "openai-codex",
    "model": "gpt-5.4",
    "attempt_count": 1,
    "used_failover": false
  }
]
}
```

如果 agent 未使用 `chatgpt_oauth` provider 或未配置 pool, 则 `pool_providers` 为空数组, `provider_counts` / `recent_requests` 也为空。

追踪存储不可用时返回 `503`。

唤醒（外部触发）

POST `/v1/agents/{id}/wake`

```
{
  "message": "Process new data",
  "session_key": "optional-session",
  "user_id": "optional-user",
  "metadata": {}
}
```

响应: `{content, run_id, usage?}`。由编排工具（n8n、Paperclip）用于从外部触发 agent 运行。

Provider

GET `/v1/providers`

列出所有 LLM provider。

POST `/v1/providers`

创建 LLM provider。

```
curl -X POST http://localhost:18790/v1/providers \
-H "Authorization: Bearer TOKEN" \
-H "Content-Type: application/json" \
```


- 快速入门
- 什么是 GoClaw?
- 安装
- 快速开始
- 配置
- Web Dashboard 导航
- 从 OpenClaw 迁移

- 核心概念
- GoClaw 工作原理
- Agent 详解
- Session 与历史记录
- Tools 概览
- Memory 系统
- 多租户

- AGENT
- 创建 Agent
- Open vs. Predefined Agent
- Context 文件
- Summoning 与 Bootstrap
- 编辑 Agent Personality
- 共享与访问控制
- 用户覆盖
- System Prompt 结构

- PROVIDER
- Provider 概览
- Anthropic (Claude)
- OpenAI / Azure OpenAI
- OpenRouter
- Google Gemini
- DeepSeek
- Groq
- Mistral
- xAI (Grok)
- MiniMax
- Cohere
- Perplexity
- DashScope (阿里通义千问)
- Vertex AI (Google Cloud)
- 百炼
- Zai
- Claude CLI
- Codex / ChatGPT (OAuth)
- Ollama
- Ollama Cloud
- Suno
- YesScale
- Novita AI

`status` 为 `reconnected`（已在内存中重新注册）、`not_registered`（已启用但注册失败）或 `disabled`（provider 已禁用）。会发出 `cache_invalidate` 事件，使已连接的 agent 获取该变更。

GET /v1/providers/{id}/codex-pool-activity

返回 provider 级别的 Codex OAuth pool 路由活动（另见上方 agent 级别端点）。

GET /v1/embedding/status

检查 embedding 是否已配置并在各 provider 中可用。

GET /v1/providers/claude-cli/auth-status

检查 Claude CLI 认证状态（全局，非按 provider）。

Skill

GET /v1/skills

列出所有 skill。

POST /v1/skills/upload

以 `.zip` 文件上传 skill（最大 20 MB）。

```
curl -X POST http://localhost:18790/v1/skills/upload \
-H "Authorization: Bearer TOKEN" \
-F "file=@my-skill.zip"
```

GET /v1/skills/{id}

获取 skill 元数据。

PUT /v1/skills/{id}

更新 skill 元数据。

DELETE /v1/skills/{id}

删除 skill。

POST /v1/skills/{id}/toggle

切换 skill 启用/禁用状态。

PUT /v1/skills/{id}/tenant-config

为 skill 设置租户级覆盖（如为当前租户启用/禁用）。仅管理员。

DELETE /v1/skills/{id}/tenant-config

移除租户级覆盖（恢复默认值）。仅管理员。

Skills 导出 / 导入

以 tar.gz 归档格式导出和导入自定义 skill。

方法	路径	说明
GET	/v1/skills/export/preview	预览导出数量
GET	/v1/skills/export	直接下载 skills 归档（tar.gz）
POST	/v1/skills/import	导入 skills 归档（multipart 字段 <code>file</code> ）

导出查询参数：

- 快速入门
- 什么是 GoClaw?
- 安装
- 快速开始
- 配置
- Web Dashboard 导览
- 从 OpenClaw 迁移
- 核心概念
- GoClaw 工作原理
- Agent 详解
- Session 与历史记录
- Tools 概览
- Memory 系统
- 多租户

- AGENT
- 创建 Agent
- Open vs. Predefined Agent
- Context 文件
- Summoning 与 Bootstrap
- 编辑 Agent Personality
- 共享与访问控制
- 用户覆盖
- System Prompt 结构
- PROVIDER
- Provider 概览
- Anthropic (Claude)
- OpenAI / Azure OpenAI
- OpenRouter
- Google Gemini
- DeepSeek
- Groq
- Mistral
- xAI (Grok)
- MiniMax
- Cohere
- Perplexity
- DashScope (阿里通义千问)
- Vertex AI (Google Cloud)
- 百炼
- Zai
- Claude CLI
- Codex / ChatGPT (OAuth)
- Ollama
- Ollama Cloud
- Suno
- YesScale
- Novita AI

`stream` `bool` 为 `true` 时以 SSE 流式推送进度，最后发送 `download_url` 的 `complete` 事件

归档格式 (`skills-YYYYMMDD.tar.gz`) :

- `skills/{slug}/metadata.json` – skill 元数据 (name、slug、visibility、tags)
- `skills/{slug}/SKILL.md` – skill 文件内容
- `skills/{slug}/grants.jsonl` – agent grant (agent_key + pinned version)

导入响应 (`201 Created`) :

```
{
  "skills_imported": 3,
  "skills_skipped": 1,
  "grants_applied": 5
}
```

若 slug 在该租户中已存在则跳过（不覆盖）。Grant 通过 `agent_key` 引用 agent，未匹配的 key 将被静默跳过。

Skill 授权

方法	路径	说明
GET	<code>/v1/skills/{id}/grants/agent</code>	列出 skill 的 agent 授权 (admin+)
POST	<code>/v1/skills/{id}/grants/agent</code>	向 agent 授权 skill (body 支持 <code>can_manage</code> 标志, migration 66)
DELETE	<code>/v1/skills/{id}/grants/agent/{agentID}</code>	撤销 agent 授权
POST	<code>/v1/skills/{id}/grants/user</code>	向用户授权 skill
DELETE	<code>/v1/skills/{id}/grants/user/{userID}</code>	撤销用户授权
GET	<code>/v1/agents/{agentID}/skills</code>	列出 agent 可访问的 skill

Skill 文件与依赖

方法	路径	说明
GET	<code>/v1/skills/{id}/versions</code>	列出可用版本
GET	<code>/v1/skills/{id}/files</code>	列出 skill 中的文件
GET	<code>/v1/skills/{id}/files/{path...}</code>	读取文件内容
POST	<code>/v1/skills/rescan-deps</code>	重新扫描运行时依赖
POST	<code>/v1/skills/install-deps</code>	安装所有缺失依赖
POST	<code>/v1/skills/install-dep</code>	安装单个依赖
GET	<code>/v1/skills/runtimes</code>	检查运行时可用性

工具

直接调用

POST `/v1/tools/invoke`

- 快速入门
- 什么是 GoClaw?
- 安装
- 快速开始
- 配置
- Web Dashboard 导览

从 OpenClaw 迁移

- 核心概念
- GoClaw 工作原理

- Agent 详解
- Session 与历史记录
- Tools 概览
- Memory 系统
- 多租户

- AGENT
- 创建 Agent
- Open vs. Predefined Agent
- Context 文件
- Summoning 与 Bootstrap
- 编辑 Agent Personality
- 共享与访问控制
- 用户覆盖
- System Prompt 结构

- PROVIDER
- Provider 概览
- Anthropic (Claude)
- OpenAI / Azure OpenAI
- OpenRouter
- Google Gemini
- DeepSeek
- Groq
- Mistral
- xAI (Grok)
- MiniMax
- Cohere
- Perplexity
- DashScope (阿里通义千问)
- Vertex AI (Google Cloud)
- 百炼
- Zai
- Claude CLI
- Codex / ChatGPT (OAuth)
- Ollama
- Ollama Cloud
- Suno
- YesScale
- Novita AI

```
{
  "tool": "web_fetch",
  "action": "fetch",
  "args": {"url": "https://example.com"},
  "dryRun": false,
  "agentId": "optional",
  "channel": "optional",
  "chatId": "optional",
  "peerKind": "direct"
}
```

设置 `"dryRun": true` 可返回工具 schema 而不执行。

内置工具

方法	路径	说明
GET	<code>/v1/tools/builtin</code>	列出所有内置工具
GET	<code>/v1/tools/builtin/{name}</code>	获取工具定义
GET	<code>/v1/tools/builtin/{name}/tenant-config</code>	获取内置工具的租户级配置
PUT	<code>/v1/tools/builtin/{name}</code>	更新启用状态/设置
PUT	<code>/v1/tools/builtin/{name}/tenant-config</code>	设置租户级覆盖（管理员）
DELETE	<code>/v1/tools/builtin/{name}/tenant-config</code>	移除租户级覆盖（管理员）

注意： REST API 的自定义工具端点当前未实现。推荐使用 MCP servers 和 skills 作为扩展机制。

记忆

基于 pgvector 的按 agent 向量记忆。

方法	路径	说明
GET	<code>/v1/memory/documents</code>	全局列出所有文档
GET	<code>/v1/agents/{agentID}/memory/documents</code>	列出 agent 的文档
GET	<code>/v1/agents/{agentID}/memory/documents/{path...}</code>	获取文档详情
PUT	<code>/v1/agents/{agentID}/memory/documents/{path...}</code>	写入/更新文档
DELETE	<code>/v1/agents/{agentID}/memory/documents/{path...}</code>	删除文档
GET	<code>/v1/agents/{agentID}/memory/chunks</code>	列出文档的 chunk
POST	<code>/v1/agents/{agentID}/memory/index</code>	索引单个文档
POST	<code>/v1/agents/{agentID}/memory/index-all</code>	索引所有文档
POST	<code>/v1/agents/{agentID}/memory/search</code>	语义搜索

可选查询参数 `?user_id=` 用于按用户范围隔离。

V3 Agent 能力

v3 新增。通过 **V3 Feature Flags** 按 agent 启用。

Evolution（Agent 进化）

跟踪 tool 使用指标并接收自动改进建议。

mode 取值： **standalone**（直接处理）、**delegate**（通过 agent link 委托）、**team**（通过团队任务系统路由）

方法	路径	说明
GET	/v1/agents/{agentID}/v3-flags	获取 agent 的所有 v3 标志
PATCH	/v1/agents/{agentID}/v3-flags	更新标志（支持部分更新）

标志键：`evolution_enabled`、`episodic_enabled`、`vault_enabled`、`orchestration_enabled`、`skill_evolve`、`self_evolve`

知识图谱

按 agent 的实体-关系图谱。

方法	路径	说明
GET	/v1/agents/{agentID}/kg/entities	列出/搜索实体（BM25）
GET	/v1/agents/{agentID}/kg/entities/{entityID}	获取实体及其关系
POST	/v1/agents/{agentID}/kg/entities	更新插入实体
DELETE	/v1/agents/{agentID}/kg/entities/{entityID}	删除实体
POST	/v1/agents/{agentID}/kg/traverse	遍历图谱（最大深度 3）
POST	/v1/agents/{agentID}/kg/extract	LLM 驱动的实体提取
GET	/v1/agents/{agentID}/kg/stats	知识图谱统计
GET	/v1/agents/{agentID}/kg/graph	可视化用完整图谱
GET	/v1/agents/{agentID}/kg/graph/compact	精简图谱表示（比完整图谱 payload 更轻量）
POST	/v1/agents/{agentID}/kg/dedup/scan	扫描重复实体
GET	/v1/agents/{agentID}/kg/dedup	列出去重候选项
POST	/v1/agents/{agentID}/kg/merge	合并重复实体
POST	/v1/agents/{agentID}/kg/dedup/dismiss	忽略去重候选项

Trace

GET /v1/traces

列出 LLM traces。支持查询参数：`agentId`、`userId`、`status`、`limit`、`offset`。

```
curl "http://localhost:18790/v1/traces?agentId=UUID&limit=50" \
-H "Authorization: Bearer TOKEN"
```

GET /v1/traces/follow

按单个会话或 agent 范围轮询 trace 更新 — 专为实时"tail"视图设计。需要 `session_key` 或 `agent_id`。非管理员调用方仅限于自己的 trace。

```
curl "http://localhost:18790/v1/traces/follow?agent_id=AGENT_UUID&since=2026-06-07T08:00:00;" \
-H "Authorization: Bearer TOKEN"
```

查询参数：

参数	类型	说明
<code>session_key</code>	string	要跟踪的会话（省略 <code>agent_id</code> 时必须填）

- 快速入门
- 什么是 GoClaw?
- 安装
- 快速开始
- 配置
- Web Dashboard 导航
- 从 OpenClaw 迁移

- 核心概念
- GoClaw 工作原理
- Agent 详解
- Session 与历史记录
- Tools 概览
- Memory 系统
- 多租户

- AGENT
- 创建 Agent
- Open vs. Predefined Agent
- Context 文件
- Summoning 与 Bootstrap
- 编辑 Agent Personality
- 共享与访问控制
- 用户覆盖
- System Prompt 结构

- PROVIDER
- Provider 概览
- Anthropic (Claude)
- OpenAI / Azure OpenAI
- OpenRouter
- Google Gemini
- DeepSeek
- Groq
- Mistral
- xAI (Grok)
- MiniMax
- Cohere
- Perplexity
- DashScope (阿里通义千问)
- Vertex AI (Google Cloud)
- 百炼
- Zai
- Claude CLI
- Codex / ChatGPT (OAuth)
- Ollama
- Ollama Cloud
- Suno
- YesScale
- Novita AI

agent_id	string	要跟踪的 agent（省略 session_key 时必填）
status	string	按状态过滤
channel	string	按 channel 过滤
since	RFC 3339	仅返回此时间之后变更的 trace
limit	integer	trace 数上限（默认 50，最大 200）
include_spans	boolean	是否包含 span 详情（默认 false）

GET /v1/traces/{traceID}

获取单条 trace 及其所有 span。

GET /v1/traces/{traceID}/export

将 trace 树导出为 gzip 压缩的 JSON。

成本

方法	路径	说明
GET	/v1/costs/summary	按 agent/时间范围统计成本

用量与分析

方法	路径	说明
GET	/v1/usage/timeseries	时序用量数据点
GET	/v1/usage/breakdown	按 provider/model/channel 分类
GET	/v1/usage/summary	含环比对比的摘要

查询参数： from 、 to （RFC 3339） 、 agent_id 、 provider 、 model 、 channel 、 group_by

用量上限（Usage Caps）

限制每个时间窗口内 token 用量和成本的支出上限策略，可选择按特定 agent、provider 或 model 限定范围。所有端点都需要**管理员角色**；写操作还需要**租户管理员（tenant-admin）**。被标记为 *managed*（由平台预置）的策略无法编辑或删除（ 409 ）。

方法	路径	说明
GET	/v1/usage-caps/policies	列出当前租户的支出上限策略
POST	/v1/usage-caps/policies	创建策略
PATCH	/v1/usage-caps/policies/{id}	部分更新（所有字段可选）
DELETE	/v1/usage-caps/policies/{id}	删除策略（ 204 ）
GET	/v1/usage-caps/utilization	各策略的当前用量
GET	/v1/usage-caps/events	近期上限事件（限流、超限）

POST /v1/usage-caps/policies

```
curl -X POST http://localhost:18790/v1/usage-caps/policies \
-H "Authorization: Bearer TOKEN" \
-H "Content-Type: application/json" \
-d '{
  "window": "day",
```

- 快速入门
- 什么是 GoClaw?
- 安装
- 快速开始
- 配置
- Web Dashboard 导航
- 从 OpenClaw 迁移

- 核心概念
- GoClaw 工作原理
- Agent 详解
- Session 与历史记录
- Tools 概览
- Memory 系统
- 多租户

- AGENT
- 创建 Agent
- Open vs. Predefined Agent
- Context 文件
- Summoning 与 Bootstrap
- 编辑 Agent Personality
- 共享与访问控制
- 用户覆盖

- PROVIDER
- Provider 概览
- Anthropic (Claude)
- OpenAI / Azure OpenAI
- OpenRouter
- Google Gemini
- DeepSeek
- Groq
- Mistral
- xAI (Grok)
- MiniMax
- Cohere
- Perplexity
- DashScope (阿里通义千问)
- Vertex AI (Google Cloud)
- 百炼
- Zai
- Claude CLI
- Codex / ChatGPT (OAuth)
- Ollama
- Ollama Cloud
- Suno
- YesScale
- Novita AI

```
"max_tokens": 1000000,  
"max_cost_usd": 25.0,  
"agent_id": null,  
"provider_id": null,  
"priority": 100,  
"enabled": true  
}'
```

字段	类型	说明
<code>window</code>	string	重置窗口 — 如 <code>day</code> （默认）。必填
<code>max_tokens</code>	integer	每窗口 token 上限（可选）
<code>max_cost_usd</code>	float	成本上限（USD，可选；转换为 micros）
<code>max_cost_micros</code>	integer	成本上限（微美元）— 优先于 <code>max_cost_usd</code> （可选）
<code>agent_id</code>	UUID	限定为单个 agent（可选）
<code>provider_id</code>	UUID	限定为单个 provider（可选）
<code>provider_type</code>	string	限定为某 provider 类型（可选）
<code>model_id</code>	string	限定为单个 model（可选）
<code>priority</code>	integer	评估优先级，越高越先（默认 100）
<code>enabled</code>	boolean	默认 <code>true</code>

返回 `201` 及创建的策略。`PATCH` 接受相同字段，均可选 — 对 `max_tokens` / `max_cost_*` 发送 `null` 以清除上限。

`GET /v1/usage-caps/events` 接受 `?limit=`，返回 `{ "events": [...] }`。Utilization 返回 `{ "rows": [...] }`。

模型定价（Model Pricing）

按模型的定价目录及按租户的覆盖（override），用于计算用量成本。读取端点需要**管理员角色**；写入覆盖需要**租户管理员**；OpenRouter 同步需要**master 范围**。

方法	路径	说明
<code>GET</code>	<code>/v1/model-pricing</code>	列出定价目录
<code>GET</code>	<code>/v1/model-pricing/overrides</code>	列出租户定价覆盖
<code>PUT</code>	<code>/v1/model-pricing/overrides</code>	upsert 一条定价覆盖
<code>DELETE</code>	<code>/v1/model-pricing/overrides/{id}</code>	移除一条覆盖
<code>POST</code>	<code>/v1/model-pricing/sync-openrouter</code>	从 OpenRouter 刷新目录（master 范围）

`GET /v1/model-pricing` 查询参数：`model`（按 model ID 过滤）、`limit`（默认 100）。响应 `{ "models": [...] }`。

`GET /v1/model-pricing/overrides` 查询参数：`provider_id`（可选 UUID 过滤）。响应 `{ "overrides": [...] }`。

`PUT /v1/model-pricing/overrides`

```
curl -X PUT http://localhost:18790/v1/model-pricing/overrides \  
-H "Authorization: Bearer TOKEN" \  
-H "Content-Type: application/json" \  
-d '{  
  "provider_id": "PROVIDER_UUID",  
  "provider_type": "openai_compat",  
  "model_id": "gpt-4o",  
  "pricing": { "...": "..." },  
'
```

"enabled": true
}'

Home

GitHub

字段	类型	说明
provider_id	UUID	必填
provider_type	string	provider 类型
model_id	string	必填
pricing	object	定价字段（输入/输出费率等）
enabled	boolean	默认 <code>true</code>

POST /v1/model-pricing/sync-openrouter 拉取 OpenRouter 目录并 upsert。响应 `{ "count": N }`。若上游拉取失败返回 `502`。

浏览器 Cookie 同步（Browser Cookie Sync）

存储选定的浏览器 cookie，使服务端浏览器会话（如 browser 工具）能复用已登录状态。所有端点都需要 **operator** 角色。Cookie 按租户 + 用户（+ 可选 agent）限定范围并在静态存储时加密 — 必须设置 `GOCLAW_ENCRYPTION_KEY`，否则 sync 返回 `503`。

方法	路径	说明
POST	/v1/browser/cookies/sync	为当前范围 upsert cookie
GET	/v1/browser/cookies	列出已存储的 cookie 元数据（从不返回值）
DELETE	/v1/browser/cookies	删除当前范围内已存储的 cookie

POST /v1/browser/cookies/sync

```
curl -X POST http://localhost:18790/v1/browser/cookies/sync \  
-H "Authorization: Bearer TOKEN" \  
-H "X-GoClaw-User-Id: user123" \  
-H "Content-Type: application/json" \  
-d '{  
  "agent_id": "AGENT_UUID",  
  "source": "chrome-extension",  
  "cookies": [  
    {  
      "domain": "example.com",  
      "name": "session",  
      "path": "/",  
      "value": "abc123",  
      "secure": true,  
      "httpOnly": true,  
      "sameSite": "Lax",  
      "expiresAt": "2026-12-31T00:00:00Z"  
    }  
  ]  
}'
```

字段	类型	说明
agent_id	string	可选的 agent 范围（别名： <code>agent</code> ）
source	string	来源标签（默认 <code>chrome-extension</code> ）
cookies	array	cookie 项。必填、非空、最多 200 项

cookie 项字段： `domain`（或用 `url` 推导）、`name`、`path`、`value`（≤16 KB）、`secure`、`httpOnly`（别名 `http_only`）、`sameSite`（别名 `same_site`），以及 `expiresAt` / `expires_at` / `expirationDate` / `expiration_date` 之一。整个请求体上限为 1 MB。

响应： `{ "synced": N }`。

- 快速入门
- 什么是 GoClaw?
- 安装
- 快速开始
- 配置
- Web Dashboard 导航
- 从 OpenClaw 迁移

- 核心概念
- GoClaw 工作原理
- Agent 详解
- Session 与历史记录
- Tools 概览
- Memory 系统
- 多租户
- AGENT
- 创建 Agent
- Open vs. Predefined Agent
- Context 文件
- Summoning 与 Bootstrap
- 编辑 Agent Personality
- 共享与访问控制
- 用户覆盖
- System Prompt 结构

- PROVIDER
- Provider 概览
- Anthropic (Claude)
- OpenAI / Azure OpenAI
- OpenRouter
- Google Gemini
- DeepSeek
- Groq
- Mistral
- xAI (Grok)
- MiniMax
- Cohere
- Perplexity
- DashScope (阿里通义千问)
- Vertex AI (Google Cloud)
- 百炼
- Zai
- Claude CLI
- Codex / ChatGPT (OAuth)
- Ollama
- Ollama Cloud
- Suno
- YesScale
- Novita AI

仅返回 cookie 元数据 — 值从不暴露。可选查询过滤: agent_id, domain, name, path, 响应

```
"items": [ { "domain", "name", "path", "secure", "httpOnly", "sameSite", "expiresAt", "source", "updatedAt" } ] }
```

DELETE /v1/browser/cookies

删除当前范围内的 cookie。与 list 端点相同的可选过滤。响应 { "deleted": N }。

MCP Server

GET /v1/mcp/servers

列出所有 MCP server 配置。

POST /v1/mcp/servers

注册 MCP server。

```
curl -X POST http://localhost:18790/v1/mcp/servers \
-H "Authorization: Bearer TOKEN" \
-H "Content-Type: application/json" \
-d '{
  "name": "filesystem",
  "transport": "stdio",
  "command": "npx",
  "args": ["-y", "@modelcontextprotocol/server-filesystem", "/tmp"],
  "enabled": true
}'
```

传输选项: "stdio"、"sse"、"streamable-http"。

GET /v1/mcp/servers/{id}

获取 MCP server。

PUT /v1/mcp/servers/{id}

更新 MCP server。可更新字段:

字段	类型	说明
name	string	Server 显示名称
transport	string	"stdio"、"sse"、"streamable-http"
command	string	运行命令 (stdio)
args	string[]	命令参数
url	string	Server URL (sse/streamable-http)
api_key	string	Server 的 API key
env	object	环境变量
headers	object	HTTP 请求头
enabled	boolean	启用/禁用
tool_prefix	string	工具名称前缀
timeout_sec	integer	请求超时 (秒)
agent_id	string	绑定到特定 agent
config	object	额外配置
settings	object	Server 设置

DELETE /v1/mcp/servers/{id}

- 快速入门
- 什么是 GoClaw?
- 安装
- 快速开始
- 配置
- Web Dashboard 导览
- 从 OpenClaw 迁移

- 核心概念
- GoClaw 工作原理
- Agent 详解
- Session 与历史记录
- Tools 概览
- Memory 系统
- 多租户
- AGENT
- 创建 Agent
- Open vs. Predefined Agent
- Context 文件
- Summoning 与 Bootstrap
- 编辑 Agent Personality
- 共享与访问控制
- 用户覆盖
- System Prompt 结构

- PROVIDER
- Provider 概览
- Anthropic (Claude)
- OpenAI / Azure OpenAI
- OpenRouter
- Google Gemini
- DeepSeek
- Groq
- Mistral
- xAI (Grok)
- MiniMax
- Cohere
- Perplexity
- DashScope (阿里通义千问)
- Vertex AI (Google Cloud)
- 百炼
- Zai
- Claude CLI
- Codex / ChatGPT (OAuth)
- Ollama
- Ollama Cloud
- Suno
- YesScale
- Novita AI

删除 MCP server。

`POST /v1/mcp/servers/test`

保存前测试 MCP server 连通性。

`POST /v1/mcp/servers/{id}/reconnect`

强制重新连接运行中的 MCP server。

`GET /v1/mcp/servers/{id}/tools`

列出运行中的 MCP server 发现的工具。

MCP 授权

方法	路径	说明
GET	<code>/v1/mcp/servers/{id}/grants</code>	列出 server 的授权记录
POST	<code>/v1/mcp/servers/{id}/grants/agent</code>	向 agent 授权 server
DELETE	<code>/v1/mcp/servers/{id}/grants/agent/{agentID}</code>	撤销 agent 授权
GET	<code>/v1/mcp/grants/agent/{agentID}</code>	列出 agent 的所有授权
POST	<code>/v1/mcp/servers/{id}/grants/user</code>	向用户授权 server
DELETE	<code>/v1/mcp/servers/{id}/grants/user/{userID}</code>	撤销用户授权

MCP 访问请求

方法	路径	说明
POST	<code>/v1/mcp/requests</code>	提交访问请求
GET	<code>/v1/mcp/requests</code>	列出待处理请求
POST	<code>/v1/mcp/requests/{id}/review</code>	批准或拒绝请求

MCP 导出 / 导入

以 tar.gz 归档格式导出和导入 MCP server 配置及 agent grant。

方法	路径	说明
GET	<code>/v1/mcp/export/preview</code>	预览导出数量（不生成归档）
GET	<code>/v1/mcp/export</code>	直接下载 MCP 归档（tar.gz）
POST	<code>/v1/mcp/import</code>	导入 MCP 归档（multipart 字段 <code>file</code> ）

MCP 用户凭证

为需要独立认证的 MCP server 提供按用户凭证存储。

方法	路径	说明
PUT	<code>/v1/mcp/servers/{id}/user-credentials</code>	为 server 设置用户凭证
GET	<code>/v1/mcp/servers/{id}/user-credentials</code>	获取用户凭证
DELETE	<code>/v1/mcp/servers/{id}/user-credentials</code>	删除用户凭证

导出查询参数：

参数	类型	说明
<code>stream</code>	<code>bool</code>	为 <code>true</code> 时以 SSE 流式推送进度，最后发送含 <code>download_url</code> 的 <code>complete</code> 事件

归档格式（`mcp-servers-YYYYMMDD.tar.gz`）：

- 快速入门
- 什么是 GoClaw?
- 安装
- 快速开始
- 配置
- Web Dashboard 导览
- 从 OpenClaw 迁移

- 核心概念
- GoClaw 工作原理
- Agent 详解
- Session 与历史记录
- Tools 概览
- Memory 系统
- 多租户
- AGENT
- 创建 Agent
- Open vs. Predefined Agent
- Context 文件
- Summoning 与 Bootstrap
- 编辑 Agent Personality
- 共享与访问控制
- 用户覆盖
- System Prompt 结构

- PROVIDER
- Provider 概览
- Anthropic (Claude)
- OpenAI / Azure OpenAI
- OpenRouter
- Google Gemini
- DeepSeek
- Groq
- Mistral
- xAI (Grok)
- MiniMax
- Cohere
- Perplexity
- DashScope (阿里通义千问)
- Vertex AI (Google Cloud)
- 百炼
- Zai
- Claude CLI
- Codex / ChatGPT (OAuth)
- Ollama
- Ollama Cloud
- Suno
- YesScale
- Novita AI

导入响应 (201 Created) :

```
{
  "servers_imported": 2,
  "servers_skipped": 0,
  "grants_applied": 4
}
```

Channel 实例

GET /v1/channels/instances

列出数据库中的所有 channel 实例。

POST /v1/channels/instances

创建 channel 实例。

```
curl -X POST http://localhost:18790/v1/channels/instances \
-H "Authorization: Bearer TOKEN" \
-H "Content-Type: application/json" \
-d '{
  "name": "my-telegram-bot",
  "channel_type": "telegram",
  "agent_id": "AGENT_UUID",
  "credentials": { "token": "BOT_TOKEN" },
  "enabled": true
}'
```

支持的 channel: telegram 、 discord 、 slack 、 whatsapp 、 zalo_oa 、 zalo_personal 、 feishu

GET /v1/channels/instances/{id}

获取 channel 实例。

PUT /v1/channels/instances/{id}

更新 channel 实例。可更新字段:

字段	类型	说明
channel_type	string	Channel 类型
credentials	object	Channel 凭证
agent_id	string	绑定的 agent UUID
enabled	boolean	启用/禁用
display_name	string	人类可读名称
group_policy	string	群组消息策略
allow_from	string[]	允许的发送者 ID
metadata	object	自定义元数据
webhook_secret	string	Webhook 验证密钥
config	object	额外配置

DELETE /v1/channels/instances/{id}

删除 channel 实例。

快速入门

什么是 GoClaw?

安装

快速开始

配置

Web Dashboard 导览

从 OpenClaw 迁移

核心概念

GoClaw 工作原理

Agent 详解

Session 与历史记录

Tools 概览

Memory 系统

多租户

AGENT

创建 Agent

Open vs. Predefined Agent

Context 文件

Summoning 与 Bootstrap

编辑 Agent Personality

共享与访问控制

用户覆盖

System Prompt 结构

PROVIDER

Provider 概览

Anthropic (Claude)

OpenAI / Azure OpenAI

OpenRouter

Google Gemini

DeepSeek

Groq

Mistral

xAI (Grok)

MiniMax

Cohere

Perplexity

DashScope (阿里通义千问)

Vertex AI (Google Cloud)

百炼

Zai

Claude CLI

Codex / ChatGPT (OAuth)

Ollama

Ollama Cloud

Suno

YesScale

Novita AI

方法	路径	说明
GET	/v1/channels/instances/{id}/writers/groups	列出有写入权限的群组
GET	/v1/channels/instances/{id}/writers	列出已授权的写入者
POST	/v1/channels/instances/{id}/writers/test	测试某用户是否被允许写入某群组（不修改数据）
POST	/v1/channels/instances/{id}/writers	添加写入者
DELETE	/v1/channels/instances/{id}/writers/{userId}	移除写入者

POST .../writers/test — 在持久化前先试运行写入者检查。请求体 `{ "group_id": "...", "user_id": "..."}` （两者均必填）。始终返回 `200` 并附带评估结果：

```
{
  "allowed": true,
  "reason": "writer",
  "instance_id": "uuid",
  "agent_id": "uuid",
  "group_id": "telegram:-100123",
  "user_id": "user123",
  "writer_count": 3
}
```

`reason` 为 `writer`、`not_writer`、`no_writers_configured` 或 `invalid_group` 之一。

联系人

方法	路径	说明
GET	/v1/contacts	列出联系人（分页）
GET	/v1/contacts/resolve	按 ID 解析联系人（重复 query string <code>ids=</code> ；最多 100 个）
POST	/v1/contacts/merge	合并重复联系人记录
POST	/v1/contacts/unmerge	取消已合并的联系人
GET	/v1/contacts/merged/{tenantUserId}	列出租户用户的已合并联系人

租户用户

方法	路径	说明
GET	/v1/tenant-users	列出租户用户
GET	/v1/users/search	跨 channel 搜索用户

团队事件

方法	路径	说明
GET	/v1/teams/{id}/events	列出团队事件（分页）

团队工作区

方法	路径	说明
POST	/v1/teams/{teamId}/workspace/upload	上传文件到团队工作区
PUT	/v1/teams/{teamId}/workspace/move	移动/重命名团队工作区中的文件

快速入门

什么是 GoClaw?

安装

快速开始

配置

Web Dashboard 导览

从 OpenClaw 迁移

核心概念

GoClaw 工作原理

Agent 详解

Session 与历史记录

Tools 概览

Memory 系统

多租户

AGENT

创建 Agent

Open vs. Predefined Agent

Context 文件

Summoning 与 Bootstrap

编辑 Agent Personality

共享与访问控制

用户覆盖

System Prompt 结构

PROVIDER

Provider 概览

Anthropic (Claude)

OpenAI / Azure OpenAI

OpenRouter

Google Gemini

DeepSeek

Groq

Mistral

xAI (Grok)

MiniMax

Cohere

Perplexity

DashScope (阿里通义千问)

Vertex AI (Google Cloud)

百炼

Zai

Claude CLI

Codex / ChatGPT (OAuth)

Ollama

Ollama Cloud

Suno

YesScale

Novita AI

方法	路径	说明
GET	/v1/teams/{teamId}/attachments/{attachmentId}/download	下载任务附件

团队导出 / 导入

以 tar.gz 归档格式导出和导入完整团队（团队元数据 + 所有成员 agent）。

方法	路径	说明
GET	/v1/teams/{id}/export/preview	预览导出数量（members、tasks、agent_links），不生成归档
GET	/v1/teams/{id}/export	直接下载团队归档（tar.gz）
POST	/v1/teams/import	导入团队归档，创建新 agent 并建立团队结构（multipart 字段 file）

导出查询参数：

参数	类型	说明
stream	bool	为 true 时以 SSE 流式推送进度，最后发送含 download_url 的 complete 事件

归档格式（team-{name}-YYYYMMDD.tar.gz）：

- | | |
|-------------------------------------|---|
| manifest.json | - 归档 manifest (team_name、agent_keys、sections) |
| team/team.json | - 团队元数据 |
| team/members.jsonl | - 团队成员记录 |
| team/tasks.jsonl | - 团队任务记录 |
| team/comments.jsonl | - 任务评论 |
| team/events.jsonl | - 任务事件 |
| team/links.jsonl | - agent 链接记录 |
| team/workspace/ | - 团队工作区文件 |
| agents/{agent_key}/agent.json | - 每个 agent 的配置 |
| agents/{agent_key}/context_files/ | - 每个 agent 的 context 文件 |
| agents/{agent_key}/memory/ | - 每个 agent 的记忆文档 |
| agents/{agent_key}/knowledge_graph/ | - 每个 agent 的 KG 实体 + 关系 |
| agents/{agent_key}/cron/ | - 每个 agent 的 cron 作业 |
| agents/{agent_key}/workspace/ | - 每个 agent 的工作区文件 |

导入响应（201 Created）：

```
{  "team_name": "research-team",  "agents_added": 3,  "agent_keys": ["researcher", "writer", "reviewer"]}
```

导入需要**管理员权限**。重复的 agent key 会自动重命名（添加后缀 -2、-3）。Cron 作业始终以禁用状态导入。

通用下载端点（所有导出类型共用）：

方法	路径	说明
GET	/v1/export/download/{token}	通过短效 token 下载归档（5 分钟有效，所有导出类型共用）

待处理消息

- 快速入门
- 什么是 GoClaw?
- 安装
- 快速开始
- 配置
- Web Dashboard 导航
- 从 OpenClaw 迁移

- 核心概念
- GoClaw 工作原理
- Agent 详解
- Session 与历史记录
- Tools 概览
- Memory 系统
- 多租户
- AGENT
- 创建 Agent
- Open vs. Predefined Agent
- Context 文件
- Summoning 与 Bootstrap
- 编辑 Agent Personality
- 共享与访问控制
- 用户覆盖
- System Prompt 结构

- PROVIDER
- Provider 概览
- Anthropic (Claude)
- OpenAI / Azure OpenAI
- OpenRouter
- Google Gemini
- DeepSeek
- Groq
- Mistral
- xAI (Grok)
- MiniMax
- Cohere
- Perplexity
- DashScope (阿里通义千问)
- Vertex AI (Google Cloud)
- 百炼
- Zai
- Claude CLI
- Codex / ChatGPT (OAuth)
- Ollama
- Ollama Cloud
- Suno
- YesScale
- Novita AI

方法	路径	说明
GET	/v1/pending-messages	列出所有带标题的消息组
GET	/v1/pending-messages/messages	按 channel+key 列出消息
DELETE	/v1/pending-messages	删除消息组
POST	/v1/pending-messages/compact	基于 LLM 的摘要（异步，202）

安全 CLI 凭证

需要**管理员角色**（完整 gateway token，或开发/单用户模式下的空 gateway token）。

方法	路径	说明
GET	/v1/cli-credentials	列出所有凭证
POST	/v1/cli-credentials	创建新凭证
GET	/v1/cli-credentials/{id}	获取凭证详情
PUT	/v1/cli-credentials/{id}	更新凭证
DELETE	/v1/cli-credentials/{id}	删除凭证
GET	/v1/cli-credentials/presets	获取预设凭证模板
POST	/v1/cli-credentials/{id}/test	测试凭证连接（演习）
POST	/v1/cli-credentials/check-binary	验证 CLI 凭证的二进制路径

按用户 CLI 凭证

方法	路径	说明
GET	/v1/cli-credentials/{id}/user-credentials	列出某 CLI 配置的用户凭证
GET	/v1/cli-credentials/{id}/user-credentials/{userId}	获取用户专属凭证
PUT	/v1/cli-credentials/{id}/user-credentials/{userId}	设置用户专属凭证
DELETE	/v1/cli-credentials/{id}/user-credentials/{userId}	删除用户专属凭证

CLI 凭证 Agent 授权

按 agent 的二进制授权 — 控制哪些 agent 可使用特定 CLI 凭证二进制，可选限制参数、详细输出和超时时间。
需要 **admin 角色**。

方法	路径	说明
GET	/v1/cli-credentials/{id}/agent-grants	列出凭证的所有 agent 授权
POST	/v1/cli-credentials/{id}/agent-grants	创建 agent 授权
GET	/v1/cli-credentials/{id}/agent-grants/{grantId}	获取指定授权详情
PUT	/v1/cli-credentials/{id}/agent-grants/{grantId}	更新授权
DELETE	/v1/cli-credentials/{id}/agent-grants/{grantId}	删除授权
POST	/v1/cli-credentials/{id}/agent-grants/{grantId}/env:reveal	解密并返回授权的 env vars（有速率限制；永不缓存 — 选用 POST 而非 GET 以防 CSRF）

创建/更新授权字段：

字段	类型	说明
agent_id	UUID	被授权的 agent（创建时必须）

- 快速入门
- 什么是 GoClaw?
- 安装
- 快速开始
- 配置
- Web Dashboard 导览
- 从 OpenClaw 迁移

- 核心概念
- GoClaw 工作原理
- Agent 详解
- Session 与历史记录
- Tools 概览
- Memory 系统
- 多租户

- AGENT
- 创建 Agent
- Open vs. Predefined Agent
- Context 文件
- Summoning 与 Bootstrap
- 编辑 Agent Personality
- 共享与访问控制
- 用户覆盖
- System Prompt 结构

- PROVIDER
- Provider 概览
- Anthropic (Claude)
- OpenAI / Azure OpenAI
- OpenRouter
- Google Gemini
- DeepSeek
- Groq
- Mistral
- xAI (Grok)
- MiniMax
- Cohere
- Perplexity
- DashScope (阿里通义千问)
- Vertex AI (Google Cloud)
- 百炼
- Zai
- Claude CLI
- Codex / ChatGPT (OAuth)
- Ollama
- Ollama Cloud
- Suno
- YesScale
- Novita AI

env_vars	object	按授权覆盖 env var（输入为 plaintext；通过 GOCLAW_ENCRYPTION_KEY 加密存储；list/get 响应仅返回 key 名称）
deny_args	JSON	参数限制（可选）
deny_verbose	JSON	详细输出限制（可选）
timeout_seconds	integer	覆盖该 agent 的执行超时（可选）
tips	string	给 agent 的使用提示（可选）
enabled	boolean	启用/禁用授权（默认： <code>true</code> ）

创建响应（`201 Created`）：返回已创建的授权对象（不含 env plaintext 值 — 仅返回 `env_keys` + `env_set`）。

env:reveal 返回授权 env vars 的解密 plaintext。端点按凭证进行速率限制（可配置 RPM + burst）并记录审计日志。需要设置 `GOCLAW_ENCRYPTION_KEY`；否则仅在明确配置时才将 env 以未加密形式存储在磁盘上。

授权变更会在消息总线上发出 `cache_invalidate` 事件，使已连接的 agent 立即感知更新。

文字转语音（TTS）

按租户的 TTS 合成与配置。合成/测试端点需要 `RoleOperator`；配置端点需要 `RoleAdmin`。

POST /v1/tts/synthesize

使用已配置的 TTS provider 将文本转换为音频。

请求体：

```
{
  "text": "你好，世界！",
  "provider": "openai",
  "voice_id": "alloy",
  "model_id": "tts-1"
}
```

字段	类型	说明
text	string	要合成的文本。必填。最多 500 个字符。
provider	string	覆盖 provider（ <code>openai</code> 、 <code>elevenlabs</code> 、 <code>minimax</code> 、 <code>edge</code> 、 <code>gemini</code> ）。可选——默认使用租户配置的 provider。
voice_id	string	语音标识符。可选。
model_id	string	模型标识符。可选。

响应： 原始音频字节，`Content-Type` 与 provider 的 MIME 类型匹配（例如 `audio/mpeg`）。

错误： `400` 文本为空或超限 · `404` 未配置 provider · `422` 模型或参数无效 · `429` 频率限制 · `504` 合成超时

POST /v1/tts/test-connection

使用提供的凭证测试 TTS provider 连通性（不持久化配置）。支持与 synthesize 相同的 provider 集。传入 `***` 作为 `api_key` 可复用已保存的密钥。

请求体：

```
{
  "provider": "openai",
  "api_key": "sk-...",
  "api_base": "",
  "voice_id": "alloy",
  "model_id": "tts-1",
  "group_id": "",
}
```

快速入门

什么是 GoClaw?

安装

快速开始

配置

Web Dashboard 导览

从 OpenClaw 迁移

核心概念

GoClaw 工作原理

Agent 详解

Session 与历史记录

Tools 概览

Memory 系统

多租户

AGENT

创建 Agent

Open vs. Predefined Agent

Context 文件

Summoning 与 Bootstrap

编辑 Agent Personality

共享与访问控制

用户覆盖

System Prompt 结构

PROVIDER

Provider 概览

Anthropic (Claude)

OpenAI / Azure OpenAI

OpenRouter

Google Gemini

DeepSeek

Groq

Mistral

xAI (Grok)

MiniMax

Cohere

Perplexity

DashScope (阿里通义千问)

Vertex AI (Google Cloud)

百炼

Zai

Claude CLI

Codex / ChatGPT (OAuth)

Ollama

Ollama Cloud

Suno

YesScale

Novita AI

字段	类型	说明
<code>provider</code>	string	必填。可选值: <code>openai</code> 、 <code>elevenlabs</code> 、 <code>minimax</code> 、 <code>edge</code> 、 <code>gemini</code> 。
<code>api_key</code>	string	API key。 <code>edge</code> 以外的 provider 必填。传入 <code>***</code> 可复用已保存的密钥。
<code>api_base</code>	string	自定义 API 基础 URL。可选。
<code>voice_id</code>	string	语音标识符。可选。
<code>model_id</code>	string	模型标识符。可选。
<code>group_id</code>	string	MiniMax 的 group ID。 <code>minimax</code> 时必须。
<code>rate</code>	string	语速（仅 Edge TTS）。可选。
<code>timeout_ms</code>	integer	请求超时（毫秒）。可选（默认: 10 000）。
<code>params</code>	object	provider 专属参数 blob。可选。

响应:

```
{  
  "success": true,  
  "provider": "openai",  
  "latency_ms": 312  
}
```

失败时: `{"success": false, "error": "..."}`

错误: `400` 缺少必填字段 · `422` voice/model/params 无效 · `504` 测试超时 · `502` 上游错误

GET /v1/tts/capabilities

返回所有已知 TTS provider 的静态能力目录——与运行时实际配置的 provider 无关。用于在保存凭证前渲染 per-provider 参数编辑器。

响应:

```
{  
  "providers": [  
    {  
      "provider": "openai",  
      "models": ["tts-1", "tts-1-hd"],  
      "params": [  
        { "key": "speed", "type": "float", "min": 0.25, "max": 4.0, "default": 1.0 }  
      ]  
    },  
    ...  
  ]  
}
```

`params` 中每个条目包含: `key`、`type` (`string` | `float` | `int` | `bool` | `enum`)、可选的 `min` / `max` / `default` / `enum_values` , 以及可选的 `depends_on` 条件。

认证: `RoleOperator`

GET /v1/tts/config

返回当前租户的 TTS 配置。API key 以 `***` 脱敏显示。需要 `RoleAdmin` 和有效的租户上下文。

响应:

```
{  
  "provider": "openai",  
  "auto": "off",  
  "mode": "final",  
  "max_length": 1500,  
  "timeout_ms": 30000,  
}
```

- 快速入门
- 什么是 GoClaw?
- 安装
- 快速开始
- 配置
- Web Dashboard 导览
- 从 OpenClaw 迁移

- 核心概念
- GoClaw 工作原理
- Agent 详解
- Session 与历史记录
- Tools 概览
- Memory 系统
- 多租户
- AGENT
- 创建 Agent
- Open vs. Predefined Agent
- Context 文件
- Summoning 与 Bootstrap
- 编辑 Agent Personality
- 共享与访问控制
- 用户覆盖
- System Prompt 结构

- PROVIDER
- Provider 概览
- Anthropic (Claude)
- OpenAI / Azure OpenAI
- OpenRouter
- Google Gemini
- DeepSeek
- Groq
- Mistral
- xAI (Grok)
- MiniMax
- Cohere
- Perplexity
- DashScope (阿里通义千问)
- Vertex AI (Google Cloud)
- 百炼
- Zai
- Claude CLI
- Codex / ChatGPT (OAuth)
- Ollama
- Ollama Cloud
- Suno
- YesScale
- Novita AI

Home

GitHub

```
"openai": { "api_key": "***", "api_base": "", "voice": "alloy", "model": "tts-1" },
"elevenlabs": { "api_key": "***", "voice_id": "", "model_id": "" },
"edge": { "voice_id": "", "rate": "" },
"minimax": { "api_key": "***", "group_id": "", "voice_id": "", "model_id": "" },
"gemini": { "api_key": "***", "voice_id": "", "model_id": "" }
}
```

POST /v1/tts/config

保存当前租户的 TTS 配置。需要 `RoleAdmin` 。

请求体：

```
{
  "provider": "openai",
  "auto": "off",
  "mode": "final",
  "max_length": 1500,
  "timeout_ms": 30000,
  "openai": {
    "api_key": "sk-...",
    "api_base": "",
    "voice": "alloy",
    "model": "tts-1",
    "params": {}
  },
  "gemini": {
    "api_key": "...",
    "voice_id": "Aoede",
    "model_id": "gemini-2.5-flash-preview-tts",
    "speakers": "[{\"name\":\"Speaker1\",\"voice\":\"Aoede\"}]"
  }
}
```

字段	类型	说明
<code>provider</code>	string	当前使用的 TTS provider slug。
<code>auto</code>	string	自动应用模式： <code>off</code> 、 <code>final</code> 、 <code>all</code> 。
<code>mode</code>	string	合成触发方式： <code>final</code> （轮次结束）或 <code>chunk</code> （流式）。
<code>max_length</code>	integer	每次合成的最大字符数。
<code>timeout_ms</code>	integer	provider 请求超时（毫秒）。
<code>{provider}</code>	object	per-provider 配置。 <code>api_key: "***</code> 保留已存储的密钥不变。
<code>{provider}.params</code>	object	provider 专属参数 blob（根据能力 schema 验证）。
<code>gemini.speakers</code>	string	Gemini 多说话人模式的 JSON-encoded <code>[]SpeakerVoice</code> 。

响应：`{ "ok": true }`

语音（Voices）

按租户缓存的语音列表。支持 ElevenLabs 和 MiniMax。需要在 TTS 配置中为所请求的 provider 配置 API key。

方法	路径	说明
GET	<code>/v1/voices</code>	列出可用语音（从缓存响应；缓存未命中时实时拉取）
POST	<code>/v1/voices/refresh</code>	清除语音缓存并重新拉取。需要管理员角色。

查询参数（`GET /v1/voices`）：

参数	类型	说明
<code>provider</code>	string	语音 provider： <code>elevenlabs</code> （默认）或 <code>minimax</code> 。

- 快速入门
- 什么是 GoClaw?
- 安装
- 快速开始
- 配置
- Web Dashboard 导览
- 从 OpenClaw 迁移

- 核心概念
- GoClaw 工作原理
- Agent 详解
- Session 与历史记录
- Tools 概览
- Memory 系统
- 多租户

- AGENT
- 创建 Agent
- Open vs. Predefined Agent
- Context 文件
- Summoning 与 Bootstrap
- 编辑 Agent Personality
- 共享与访问控制
- 用户覆盖
- System Prompt 结构

- PROVIDER
- Provider 概览
- Anthropic (Claude)
- OpenAI / Azure OpenAI
- OpenRouter
- Google Gemini
- DeepSeek
- Groq
- Mistral
- xAI (Grok)
- MiniMax
- Cohere
- Perplexity
- DashScope (阿里通义千问)
- Vertex AI (Google Cloud)
- 百炼
- Zai
- Claude CLI
- Codex / ChatGPT (OAuth)
- Ollama
- Ollama Cloud
- Suno
- YesScale
- Novita AI

方法	路径	说明
GET	/v1/packages/updates	列出各 registry 发现的待更新包
POST	/v1/packages/updates/refresh	强制刷新更新 registry 缓存
POST	/v1/packages/update	应用单个更新 — body { "package": "pip:pandas" }
POST	/v1/packages/updates/apply-all	应用所有待更新包（长时任务；?stream=true 时返回 SSE 进度）

更新会在总线上发出 cache_invalidate 事件，使已连接的客户端刷新包列表。

GET /v1/shell-deny-groups

列出 shell 命令拒绝组（安全策略）。

存储

工作区文件管理。

方法	路径	说明
GET	/v1/storage/files	列出文件（支持深度限制）
GET	/v1/storage/files/{path...}	读取文件（JSON 或原始格式）
POST	/v1/storage/files	上传文件到工作区（管理员）
DELETE	/v1/storage/files/{path...}	删除文件/目录
PUT	/v1/storage/move	移动/重命名文件或目录（管理员）
GET	/v1/storage/size	流式传输存储大小（SSE，缓存 60 分钟）

?raw=true ——以原生 MIME 类型提供。?depth=N ——限制遍历深度。

媒体

方法	路径	说明
POST	/v1/media/upload	上传文件（multipart，50 MB 限制）
GET	/v1/media/{id}	按 ID 提供媒体（带缓存）

通过 Bearer token 或 ?token= 查询参数认证（用于 和 <audio> 标签）。

文件

方法	路径	说明
GET	/v1/files/{path...}	按路径提供工作区文件
POST	/v1/files/sign	生成文件访问的签名 URL

查询参数：

参数	类型	说明
download	bool	为 true 时强制 Content-Disposition: attachment（浏览器下载而非内联显示）

方法	路径	说明
GET	/v1/tenants/{id}/users	列出租户用户
POST	/v1/tenants/{id}/users	将用户添加到租户
DELETE	/v1/tenants/{id}/users/{userId}	从租户移除用户

备份与恢复

系统备份（管理员）

用于灾难恢复的全系统备份。需要管理员权限。

方法	路径	说明
POST	/v1/system/backup	触发系统备份（返回 archive 或 SSE 进度）
GET	/v1/system/backup/preflight	检查备份前置条件
GET	/v1/system/backup/download/{token}	通过短期 token 下载备份 archive

系统恢复（管理员）

方法	路径	说明
POST	/v1/system/restore	从备份 archive 恢复租户/系统。需要管理员权限。

系统备份 S3

方法	路径	说明
GET	/v1/system/backup/s3/config	获取 S3 备份配置
PUT	/v1/system/backup/s3/config	更新 S3 备份配置
GET	/v1/system/backup/s3/list	列出 S3 可用备份
POST	/v1/system/backup/s3/upload	将本地备份上传到 S3
POST	/v1/system/backup/s3/backup	直接触发备份到 S3

租户备份

按租户备份和恢复。需要管理员权限。

方法	路径	说明
POST	/v1/tenant/backup	触发租户备份
GET	/v1/tenant/backup/preflight	检查租户备份前置条件
GET	/v1/tenant/backup/download/{token}	通过短期 token 下载租户备份 archive
POST	/v1/tenant/restore	从备份 archive 恢复租户

活动与审计

方法	路径	说明
GET	/v1/activity	列出活动审计日志（可过滤）
GET	/v1/activity/aggregate	按某一维度分组的活动聚合计数
GET	/v1/activity/aggregate	

快速入门

什么是 GoClaw?

安装

快速开始

配置

Web Dashboard 导览

从 OpenClaw 迁移

核心概念

GoClaw 工作原理

Agent 详解

Session 与历史记录

Tools 概览

Memory 系统

多租户

AGENT

创建 Agent

Open vs. Predefined Agent

Context 文件

Summoning 与 Bootstrap

编辑 Agent Personality

共享与访问控制

用户覆盖

System Prompt 结构

PROVIDER

Provider 概览

Anthropic (Claude)

OpenAI / Azure OpenAI

OpenRouter

Google Gemini

DeepSeek

Groq

Mistral

xAI (Grok)

MiniMax

Cohere

Perplexity

DashScope (阿里通义千问)

Vertex AI (Google Cloud)

百炼

Zai

Claude CLI

Codex / ChatGPT (OAuth)

Ollama

Ollama Cloud

Suno

YesScale

Novita AI

返回按某一维度分组的活动计数，而非原始记录行。非管理员调用方被限制在自己的 actor ID；按 actor ID 分组需要管理员角色。

查询参数:

参数	类型	说明
group_by	string	分组维度 — 如 <code>action</code> 、 <code>entity_type</code> 、 <code>actor_type</code> 、 <code>actor_id</code> （仅管理员）
actor_type	string	按 actor 类型过滤
actor_id	string	按 actor ID 过滤
action	string	按 action 过滤
entity_type	string	按实体类型过滤
entity_id	string	按实体 ID 过滤
from	RFC 3339	时间窗口起点
to	RFC 3339	时间窗口终点 （必须晚于 <code>from</code> ）
limit	integer	返回的分组数上限

运行时日志（Runtime Logs）

内存中运行时日志缓冲区的聚合视图。需要**管理员角色**。

`GET /v1/logs/runtime/aggregate`

返回按 `level` 或 `source` 分组的运行时日志计数。

```
curl "http://localhost:18790/v1/logs/runtime/aggregate?group_by=level" \
-H "Authorization: Bearer TOKEN"
```

查询参数:

参数	类型	说明
group_by	string	<code>level</code> （默认）或 <code>source</code>
level	string	按日志级别过滤
source	string	按日志来源过滤
from	RFC 3339	仅统计此时间之后的条目

服务器日志的实时 tail 可通过 WebSocket 的 `logs.tail` 获得 — 参见 [WebSocket 协议](#)。

系统配置

按租户的键值配置存储。所有已认证用户可读；写入需要管理员角色。

方法	路径	说明
GET	<code>/v1/system-configs</code>	列出当前租户的所有配置项
GET	<code>/v1/system-configs/{key}</code>	按 key 获取单个配置值
PUT	<code>/v1/system-configs/{key}</code>	设置配置值 （仅管理员）
DELETE	<code>/v1/system-configs/{key}</code>	删除配置项 （仅管理员）

Edition

快速入门
什么是 GoClaw?
安装
快速开始
配置
Web Dashboard 导览
从 OpenClaw 迁移
核心概念
GoClaw 工作原理
Agent 详解
Session 与历史记录
Tools 概览
Memory 系统
多租户
AGENT
创建 Agent
Open vs. Predefined Agent
Context 文件
Summoning 与 Bootstrap
编辑 Agent Personality
共享与访问控制
用户覆盖
System Prompt 结构
PROVIDER
Provider 概览
Anthropic (Claude)
OpenAI / Azure OpenAI
OpenRouter
Google Gemini
DeepSeek
Groq
Mistral
xAI (Grok)
MiniMax
Cohere
Perplexity
DashScope (阿里通义千问)
Vertex AI (Google Cloud)
百炼
Zai
Claude CLI
Codex / ChatGPT (OAuth)
Ollama
Ollama Cloud
Suno
YesScale
Novita AI

方法	路径	说明
GET	/v1/workstations	列出当前租户的 workstation
POST	/v1/workstations	创建 workstation
GET	/v1/workstations/{id}	按 ID 获取 workstation
PUT	/v1/workstations/{id}	更新 workstation
DELETE	/v1/workstations/{id}	删除 workstation
POST	/v1/workstations/{id}/test	测试连通性 (SSH 探测或 docker ps) — 不执行命令
GET	/v1/workstations/{id}/permissions	列出白名单模式
POST	/v1/workstations/{id}/permissions	添加白名单模式
DELETE	/v1/workstations/{id}/permissions/{permId}	删除某个模式
PUT	/v1/workstations/{id}/permissions/{permId}/toggle	启用/禁用模式而不删除
GET	/v1/workstations/{id}/activity	列出活动审计条目 (可过滤、分页)

创建 body:

```
{
  "workstation_key": "build-host",
  "name": "Build host",
  "backend_type": "ssh",
  "metadata": {
    "host": "build.internal",
    "port": 22,
    "user": "claw",
    "private_key_ref": "kms:// ..."
  },
  "default_cwd": "/srv/builds",
  "default_env": {"PATH": "/usr/local/bin:/usr/bin"}
}
```

字段	类型	说明
workstation_key	string	面向客户端的稳定 key — 按租户唯一
name	string	显示名称
backend_type	enum	ssh 或 docker (DB CHECK 约束)
metadata	object	按 backend 的连接配置 — 加密存储
default_cwd	string	exec 调用的默认工作目录
default_env	object	合并到每次 exec 的 env var — 加密存储
active	boolean	workstation 可供 agent 链接选择

活动日志条目:

- 快速入门
- 什么是 GoClaw?
- 安装
- 快速开始
- 配置
- Web Dashboard 导航
- 从 OpenClaw 迁移

- 核心概念
- GoClaw 工作原理
- Agent 详解
- Session 与历史记录
- Tools 概览
- Memory 系统
- 多租户
- AGENT
- 创建 Agent
- Open vs. Predefined Agent
- Context 文件
- Summoning 与 Bootstrap
- 编辑 Agent Personality
- 共享与访问控制
- 用户覆盖
- System Prompt 结构

- PROVIDER
- Provider 概览
- Anthropic (Claude)
- OpenAI / Azure OpenAI
- OpenRouter
- Google Gemini
- DeepSeek
- Groq
- Mistral
- xAI (Grok)
- MiniMax
- Cohere
- Perplexity
- DashScope (阿里通义千问)
- Vertex AI (Google Cloud)
- 百炼
- Zai
- Claude CLI
- Codex / ChatGPT (OAuth)
- Ollama
- Ollama Cloud
- Suno
- YesScale
- Novita AI

action	enum	exec（已执行）或 deny（被白名单拒绝）
cmd_hash	string	完整命令行的 SHA-256
cmd_preview	string	命令的前 200 字节（截断，不含 secret）
exit_code	integer	进程退出码（ deny 时为 null）
duration_ms	integer	实际耗时（ deny 时为 null）
agent_id	UUID	发起调用的 agent

Workstation backend 共享按租户的速率限制，并遵循全局 shell deny-groups 列表。

Webhooks

两种形式：

Inbound webhook 端点 — 接收来自外部系统的 HMAC 签名请求，并通过 agent 或 channel 路由。无需 bearer token；通过 `X-GoClaw-Signature`（HMAC-SHA256）+ IP 白名单 + 可选 `localhost_only` 门控进行认证。

Webhook 管理 — 对 webhook registry 进行 CRUD 操作。需要 admin 角色和 gateway token。

Inbound webhook 需要设置 `GOCLAW_ENCRYPTION_KEY` 以便加密存储 secret。见 [环境变量](#)。

Inbound Receivers

方法	路径	说明
POST	<code>/v1/webhooks/llm</code>	LLM 类 webhook 入口（OpenAI 兼容 payload，路由到已配置的 agent）
POST	<code>/v1/webhooks/message</code>	Message 类 webhook 入口（投递到 channel/chat）

两个 receiver 均需以下请求头：

请求头	用途
<code>X-GoClaw-Webhook-Id</code>	Webhook UUID
<code>X-GoClaw-Signature</code>	<code>sha256=...</code> 用 webhook signing key 对 raw body 的 HMAC
<code>X-GoClaw-Timestamp</code>	RFC3339 时间戳（偏差 ≤ 5 分钟）
<code>Idempotency-Key</code>	可选 — 对重复投递去重（TTL 存储在 <code>webhook_calls</code> ）

Mode 通过 query string `?mode=sync|async` 选择（`llm` 默认 `sync`，`message` 默认 `async`）。

Webhook 管理

方法	路径	说明
GET	<code>/v1/webhooks</code>	列出当前租户的 webhook
POST	<code>/v1/webhooks</code>	创建 webhook — 响应包含一次性 <code>secret</code> + <code>hmac_signing_key</code>
GET	<code>/v1/webhooks/{id}</code>	获取 webhook（创建后 <code>secret</code> 不再返回）
PATCH	<code>/v1/webhooks/{id}</code>	更新可变字段（scopes、rate limits、allowlist、flags）
POST	<code>/v1/webhooks/{id}/rotate</code>	轮换 signing key — 仅返回一次新 secret
DELETE	<code>/v1/webhooks/{id}</code>	撤销 webhook（软删除；设置 <code>revoked=true</code> ）

MCP Bridge

- 快速入门
- 什么是 GoClaw?
- 安装
- 快速开始
- 配置
- Web Dashboard 导航
- 从 OpenClaw 迁移

- 核心概念
- GoClaw 工作原理
- Agent 详解
- Session 与历史记录
- Tools 概览
- Memory 系统
- 多租户

- AGENT
- 创建 Agent
- Open vs. Predefined Agent
- Context 文件
- Summoning 与 Bootstrap
- 编辑 Agent Personality
- 共享与访问控制
- 用户覆盖
- System Prompt 结构

- PROVIDER
- Provider 概览
- Anthropic (Claude)
- OpenAI / Azure OpenAI
- OpenRouter
- Google Gemini
- DeepSeek

- Groq
- Mistral
- xAI (Grok)
- MiniMax
- Cohere
- Perplexity
- DashScope (阿里通义千问)
- Vertex AI (Google Cloud)
- 百炼
- Zai
- Claude CLI
- Codex / ChatGPT (OAuth)
- Ollama
- Ollama Cloud
- Suno
- YesScale
- Novita AI

请求头	用途
X-Agent-ID	工具执行的 agent 上下文
X-User-ID	用户上下文
X-Channel	channel 路由
X-Chat-ID	聊天路由
X-Peer-Kind	direct 或 group
X-Bridge-Sig	所有 context 字段的 HMAC 签名

系统

方法	路径	说明
GET	/health	健康检查（无需认证）
GET	/v1/openapi.json	OpenAPI 3.0 规范
GET	/docs	Swagger UI

通用响应结构

成功：

```
{ "id": "uuid", "name": "...", ... }
```

错误：

```
{  "error": {    "code": "ERR_AGENT_NOT_FOUND",    "message": "Agent not found. Verify the agent ID and try again."  } }
```

错误响应使用标准化的 `envelope` 结构，包含 `code`（机器可读错误类型）和 `message`（人类可读，支持 `i18n` 翻译）。

状态码	含义
200	OK
201	Created
400	请求错误（无效 JSON、缺少字段）
401	未认证
403	禁止访问
404	未找到
409	冲突（重复名称）
429	速率限制
500	内部服务器错误

错误消息根据 `Accept-Language` 头进行本地化。

